《低多边形小岛》完整制作流程解析

> 一份写给非程序员的说明书 —— 从一行代码到能在浏览器里玩的完整游戏，中间到底发生了什么。

零、先看一眼成品

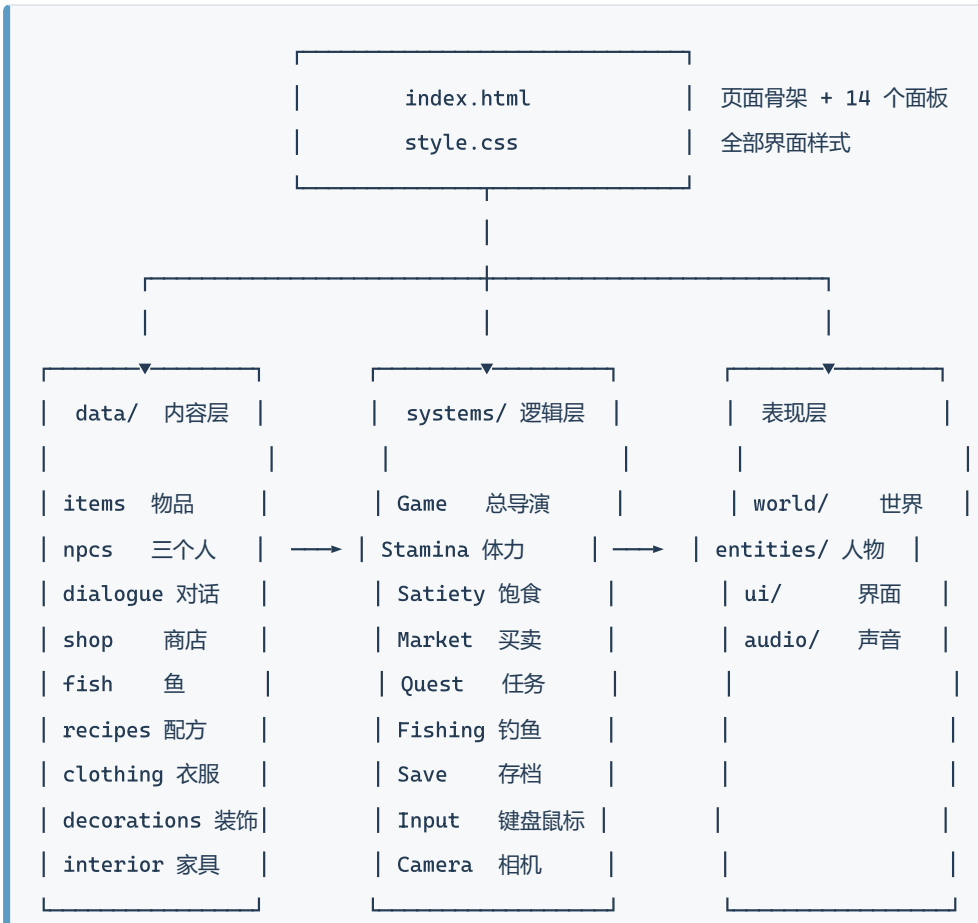
项目	数字
交付体积	单个 HTML 文件 **727 KB**（双击即玩）
源码规模	69 个 TypeScript 文件、约 **16,600 行**
其中测试	23 个文件、3,330 行、**289 个测试**
运行时依赖	**1 个**（Three.js，3D 引擎）
美术资源文件	**0 个**（没有一个 .png / .jpg / .glb / .mp3）
服务器	**不需要**（没有后端、没有数据库、没有账号）
联网需求	**不需要**（打开后不发出任何网络请求）

三个"没有"是理解这个项目的钥匙：

1. **没有美术资源文件** —— 树、房子、人、水面、天空，全部是代码在运行时算出来、拼出来的。
2. **没有服务器** —— 你的存档存在你自己的浏览器里，钱、经验、时间也全在你这台电脑上。
3. **没有安装包** —— 它就是一个网页，这也正是它能"点个链接就能玩"的原因（第九章详述）。

一、整体架构：一张图看懂

整个游戏只有三个层次，职责非常清楚：



谁是"总导演"

`src/systems/Game.ts`（2,400 多行）是整个游戏的中枢。它**自己不做任何一件具体的事**，它做的是：

- 创建渲染器、场景、相机；
- 把二十来个系统实例化并持有（体力、饱食、背包、商店、任务、钓鱼、存档、输入、相机……）；
- 每一帧按固定顺序调用它们（"先算时间，再算天气，再扣体力，再更新位置，再刷新界面"）；
- 决定"按 E 的时候到底发生什么"。

这样做的好处是：**任何玩法改动都只动一个系统文件，不会牵连别的地方。**比如"睡觉不再回饱腹度"，改的就是 `Game.ts` 里的 `restAt()` 一个方法，其它八个系统完全不知情。

一条重要纪律：逻辑和画面分开

很多游戏写成一坨（"砍树"这个函数里既改体力、又播音效、又改模型位置）。这个项目刻意避免：

- `src/world/terrain.ts` —— 地形高度是**一个纯数学函数**，文件里**故意不 import three**。所以它能在 Node 里被单元测试直接调用。
- `src/systems/*System.ts` —— 背包、商店、体力、钓鱼……全是纯逻辑，不碰画面。
- `src/world/*`、`src/entities/*` —— 只负责把逻辑的结果变成看得见的东西。

所以 `Game.ts` 是唯一"同时认识两边"的文件，它做翻译。

二、渲染模块：把数字变成画面

2.1 三件套：场景、相机、渲染器

在 `Game.ts` 的构造函数里创建：

```
this.renderer = new THREE.WebGLRenderer({ canvas, antialias: true, powerPreference: 'high-performance' });
this.renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));  // 高分屏不烧显卡
this.renderer.shadowMap.enabled = true;                              // 阴影开着
this.renderer.shadowMap.type = THREE.PCFShadowMap;                   // 软一点的阴影
this.camera = new THREE.PerspectiveCamera(CAMERA.fov, 宽高比, 0.1, 600);
```

- **WebGLRenderer**：真正把三角形画到屏幕上的东西。浏览器自带 GPU 加速，游戏本身不用写任何图形代码。
- **PerspectiveCamera（透视相机）**：人眼的看法，近大远小。`fov 62` 是视野角度，`near/far` 是能看多近多远。
- **Scene（场景）**：一个"世界容器"，所有东西挂进去，相机从里面往外看。

2.2 天空：一个自定义着色器的大球

`src/world/WorldBuilder.ts` 里，天空不是图片，是一个半径 500 米的球：

```
// 片元着色器（GLSL）：按视线方向的高度做三段渐变
float h = normalize(vWorldPosition).y;
vec3 color = mix(uHorizonColor, uTopColor, smoothstep(0.0, 0.55, h));
color = mix(uBottomColor, color, smoothstep(-0.25, 0.02, h));
```

- 顶天蓝、地平线浅蓝、地平线以下淡绿，**三段渐变**。
- `side: THREE.BackSide` 表示从球里面看（人站在球里）。
- `depthWrite: false` 表示它不写深度，永远在最远处。

2.3 地形：一个函数算出整座岛

这是整个项目最"聪明"的一块。地形**没有高度图文件**，而是一个纯函数：

```
// src/world/terrain.ts
export function heightAt(x: number, z: number): number {
  const mask = islandMask(x, z);                       // 岛的外形：中间 1，边缘 0
  const noise = signedFbm2D(x * 0.03, z * 0.03, WORLD.seed, 4);  // 起伏的噪声
  let height = BASE_ELEVATION + noise * NOISE_AMPLITUDE;
  for (const hill of HILLS) { ... }                    // 6 座山：光滑的圆形凸起
  height *= mask;                                       // 往海里沉下去
  height -= smoothstep(...) * 1.8 + ... * 16.2;         // 海底继续往下掉
  // 内湖：把地形压平成一个湖盆
  return lerp(height, -lake.depth, lakeT);
}
```

`fbm2D` 是**分形布朗运动**（fractal Brownian motion）：把不同频率的噪声叠起来，得到自然的、看不出规律的起伏。实现在 `src/utils/noise.ts`，只有 188 行，是全项目最"数学"的部分。

为什么用函数而不是图片？ 三个好处：

1. **碰撞绝对准确** —— 角色脚下的地面高度 = 直接调用 `heightAt()`，不需要射线检测，也不会出现"看着站在坡上、其实陷进地里"。
2. **世界完全确定** —— 同一个种子（`20260921`）在任何电脑、任何时候生成**完全一样**的岛。这是存档能记住"哪棵树被你砍了"的前提。
3. **改一个数字，全岛立刻变** —— 想让岛大一点？改 `WORLD.islandRadius` 就行，不用重画任何东西。

2.4 地形网格：为什么是"平面着色"

`src/world/TerrainMesh.ts` 把高度函数采样成 132×132 的网格（覆盖 264×264 米）。关键做法：

> **网格故意不用索引（non-indexed）**，这样每个三角形都拥有自己独立的三个顶点，法线也就各自独立 —— 渲染出来是一块块平整的多边形，这就是"低多边形"（low-poly）观感的来源。

颜色也是算出来的，不是贴的：每个三角形算一次自己的坡度和高度，然后从一张颜色表里插值：

```
const color = colorAt(centreX, centreZ, slope, centreY);  // 沙→草→岩石，按高度和坡度
```

材质是 `MeshLambertMaterial({ vertexColors: true, flatShading: true })` —— 兰伯特光照（便宜）+ 顶点色 + 平面着色。整座岛的颜色就来自这 7 个 RGB 常量（湿沙、沙、草低、草高、森林、岩石、岩石暗）。

2.5 水面：深度图 + 着色器

`src/world/Water.ts`：

1. 造一个 700×700 米、72×72 段的平面；
2. 采样一张 **192×192 的深度纹理**（`DataTexture`），记录每个位置的水有多深；
3. 写一个自定义 `ShaderMaterial`，用深度差判断岸边泡沫、深水颜色、波动法线。

于是水不需要任何贴图文件 —— **深度是运行时算的。**

2.6 光照与天气：同一个场景，三种情绪

`Game.applyLighting()` 根据"时段 × 天气"改四个值：太阳强度、太阳颜色、雾的颜色、雾的远近。

状态	太阳强度	太阳颜色	雾
白天晴天	1.0	暖白 `0xfff2d0`	淡蓝，100→300
白天雨天	0.55	冷灰 `0xcfd8e8`	灰蓝，60→220
傍晚	0.55	橙 `0xffb27a`	橙粉，100→300
夜晚	0.16	蓝 `0x9fb6ff`	深蓝，60→200

还有三盏固定灯：半球光（天空色+地面色，0.6）、环境光（0.22）、以及一盏跟随玩家的方向光（太阳，负责投影，阴影范围 46 米 —— 跟着玩家走，因为整座岛的阴影太贵）。

2.7 渲染循环：一秒钟跑 60 次的那段代码

```
private frame = (time: number): void => {
  if (!this.running) return;
  const dt = Math.min(0.05, Math.max(0, (time - this.lastTime) / 1000));  // 帧间隔，上限 0.05 秒
  this.elapsed += dt;
  this.update(dt);                        // 推进所有玩法
  this.renderer.render(this.scene, this.camera);   // 画一帧
  this.rafId = requestAnimationFrame(this.frame);   // 约等于每秒 60 次
};
```

`requestAnimationFrame` 是浏览器提供的回调：**页面可见时自动按屏幕刷新率触发**，标签页切到后台就自动暂停（省电）。`dt` 上限 0.05 秒是为了防止"切回来时一大跳"（例如掉帧或长时间挂起后角色瞬移）。

三、美术模块：一个图片文件都没有

3.1 低多边形 = 用基础几何体拼

Three.js 自带一批"基础体"，这个游戏全部由它们拼出来：

东西	用到的几何体
人的头/身/四肢	`BoxGeometry`（长方体）
树干	`CylinderGeometry`（圆柱，6 边）
针叶树	`ConeGeometry`（圆锥，7 边）
阔叶树冠	`IcosahedronGeometry`（二十面体）
石头	`DodecahedronGeometry`（十二面体）
灌木	`IcosahedronGeometry`
芦苇	`ConeGeometry`（开口圆锥）
草叶	手写的 `BufferGeometry`（几片三角面拼）
花	`CylinderGeometry` + `OctahedronGeometry`
贝壳	`SphereGeometry`（切掉一半）
水晶	`OctahedronGeometry`

边数越少 = 顶点越少 = 跑得越快，视觉上也就越"低多边形"。

3.2 顶点色代替贴图

没有一张图片，所以颜色怎么来？**直接写进顶点。** `src/world/PropBuilder.ts` 里的 `withColor()`：

```
function withColor(geometry: THREE.BufferGeometry, color: number): THREE.BufferGeometry {
  const count = geometry.getAttribute('position').count;
  const colors = new Float32Array(count * 3);
  const c = new THREE.Color(color);
  for (let i = 0; i < count; i++) { colors[i*3] = c.r; colors[i*3+1] = c.g; colors[i*3+2] = c.b; }
  geometry.setAttribute('color', new THREE.BufferAttribute(colors, 3));
  return geometry;
}
```

一个树干一个颜色、树冠一个颜色，材质用 `vertexColors: true` 即可。

3.3 道具工厂：`PropBuilder.ts`

所有可重复使用的物件都从这里出，函数名一看就懂：`createConifer`（针叶树）、`createBroadleaf`（阔叶树）、`createRock`、`createBush`、`createReed`、`createGrassTuft`、`createFlower`、`createShellProp`、`createCrystal`。

3.4 实例化：几百棵树只花一次绘制调用

岛上大约有几百棵树、几百丛草、几十块石头。逐个建模会卡死。解法是 **InstancedMesh**（实例化网格）：

```
// src/world/ResourceField.ts
const mesh = new THREE.InstancedMesh(part.geometry, material, list.length);
mesh.setMatrixAt(part.index, this.matrix);   // 每一棵只改一次"位置+大小+朝向"
```

GPU 只需要**画一次**树这个形状，然后重复几百次。几百个道具的渲染开销，等于一个。草也是同一招（`ResourceField.addGrass`）。

3.5 建筑与家具

`src/world/Structures.ts` 负责：地基、小屋、工作台、装饰（火堆/灯笼/彩旗/花盆/木凳）、家具（床、桌、椅、柜……）、钓鱼点。`src/world/ResourceField.ts` 负责所有**可采集资源**（树、石头、芦苇、浆果、花、贝壳、深海螺、水晶）的分布、采集与恢复。

资源分布用 `scatter()` —— 按种子把物件撒在地图上，同时保证"村里 9 米内不长树"、"深海螺只在 2.3~5.6 米深的水里"这类规则。

3.6 配色表在哪里

所有美术配色集中在 `src/data/` 里，改配色不用碰逻辑：

- `clothing.ts` —— 9 件衣服的图标、描述、价格、**加成数值**；
- `npcs.ts` —— 三个村民各自的肤色、衣色、发型色、性格、台词；
- `items.ts` —— 每个物品的名字、图标、用途。

四、人物动作：没有骨骼文件，全是每帧算出来的

这是很多人以为"必须有 3D 建模软件"的地方，其实**完全不需要**。

4.1 骨架：盒子 + 父子层级

`src/entities/CharacterModel.ts` 用长方体拼出一个人，并用 `THREE.Group`（空节点）做"关节"：

```
root（整体）
└── torso（躯干，可整体前后倾）
    ├── head（头，可左右转）
    │   ├── 头方块、头发方块、两只眼睛
    │   └── hat（帽子，买渔夫帽才显示）
    ├── leftArm（左臂，绕肩转）   → 手
    ├── rightArm（右臂）          → 手
    ├── leftLeg（左腿，绕髋转）
    ├── rightLeg（右腿）
    ├── necklace（项链）
    └── cape（斗篷 / 外套）
```

关键在于层级：把 `leftLeg` 这个 Group 绕 X 轴一转，整条腿就跟着摆 —— 这就是"关节"。要加新动作，只要转这些 Group。

4.2 走路循环：一个正弦函数

`src/entities/PlayerAvatar.ts`：

```
update(dt: number, speed01: number): void {
  const target = Math.min(1, Math.max(0, speed01));           // 0=站着, 1=冲刺
  this.phase += dt * (4.2 + target * 7.5);                     // 走得越快，摆得越快
  this.lean = lerp(this.lean, target * 0.12, Math.min(1, dt * 8)); // 加速时身体前倾

  const swing  = Math.sin(this.phase) * (0.15 + target * 0.75);
  const swing2 = Math.sin(this.phase + Math.PI) * (0.15 + target * 0.75);  // 差半个周期 = 反着摆

  this.rig.leftLeg.rotation.x  = swing;
  this.rig.rightLeg.rotation.x = swing2;
  this.rig.leftArm.rotation.x  = swing2 * 0.8;
  this.rig.rightArm.rotation.x = swing * 0.75... (0.8);

  const bob = Math.abs(Math.sin(this.phase)) * 0.05 * target;   // 走路的上下颠
  this.rig.root.position.y = bob;
  this.rig.torso.rotation.x = this.lean;
}
```

四条腿 arm/leg 用**相位差 π** 的两个正弦波，天然就是"左腿往前时右腿往后"。再加一点上下颠和前倾，看起来就像在走。**没有关键帧、没有动画文件、没有骨骼蒙皮。**

4.3 NPC 待机：呼吸、转头、任务标记

`src/entities/NpcAvatar.ts`：

```
const sway = Math.sin(this.phase * 1.3) * 0.05;
this.rig.torso.rotation.z = sway;                                    // 身体左右晃（呼吸）
this.rig.leftArm.rotation.x  = Math.sin(this.phase * 1.1) * 0.12 - 0.05;
this.rig.head.rotation.y = Math.sin(this.phase * 0.4) * 0.25;        // 偶尔看看四周
// 任务标记：一颗菱形，浮在头顶上下飘 + 自转 + 呼吸式缩放
this.marker.position.y = 2.25 + Math.sin(elapsed * 2.4) * 0.12;
this.marker.rotation.y = elapsed * 1.6;
this.markerBody.scale.setScalar(1 + Math.sin(elapsed * 4) * 0.08);
```

4.4 相机跟随与避障

`src/systems/CameraController.ts` —— 第三人称环绕相机：

- 鼠标移动 → `yaw`（左右）/ `pitch`（上下）→ 算出相机该在的位置；
- **阻尼**（`damp`）→ 相机不是瞬间跳到位，而是平滑追上去，不会晕；
- **不入地** → 相机高度永远 ≥ 地面高度 + 0.6，也不会沉到水面下；
- **不穿树** → 用射线-圆柱求交（`raycastAgainstCylinder`）算出最近障碍物，把相机拉到障碍前面。树越近，相机拉得越近。

4.5 换装：买了衣服就真的穿上

`setOutfit(owned)` 根据你拥有的衣物实时改外观：

买了什么	画面上的变化
游泳衣	上衣变成蓝色
涉水靴	裤子变成墨绿
渔夫帽	头上出现帽子（帽子组 `visible = true`）
贝壳项链 / 以太护符	脖子上出现珠串（护符是青色）
海藻斗篷 / 工作外套	背后出现披风（外套是棕色）

所以"服饰收藏"这个玩法奖励是**看得见的**，不是一行数字。

五、GUI 交互：界面是网页，不是引擎画的

5.1 面板清单

`index.html` 里直接写了 14 个面板（`<section>`），全部靠 CSS 显示/隐藏：

面板	id	打开方式
标题画面	`title-screen`	启动时
加载画面	`loading-screen`	启动时
对话	`dialogue`	走到 NPC 面前按 E
背包	`inventory`	Tab / I
合成	`crafting`	工作台
建造	`building`	地基
装饰	`decor`	小屋周边
小屋	`home`	走到小屋按 E
商店	`shop`	走到 NPC 面前按 E
钓鱼	`fishing`	钓上鱼时
储物	`storage`	小屋面板里点「储物」
屋内	`interior`	走进屋里
存档	`saves`	K
日志	`journal`	J
帮助	`help-panel`	F1

右上角常驻 HUD：体力条、饱食度条、天气、日期、目标、金币。

5.2 `GameUI.ts` 的职责

1,326 行，**只干一件事**：把数据变成 DOM，把点击变成回调。

- `showShop(options)` —— 传入商品列表，生成 HTML 列表项，每行带一个按钮；
- `renderInventory(entries)` —— 同样，生成物品行；
- `setStatus({...})` —— 更新体力条、饱食度条、日期、天气；
- `showDialogue(node)` —— 渲染对话气泡和选项。

5.3 回调注入：界面不直接改数据

界面不碰游戏逻辑，只"喊一声"：

```
// 在 GameUI 里声明
onSellMaterial: ((itemId: string) => void) | null = null;

// 在 Game 里接上
this.ui.onSellMaterial = (itemId) => this.sellMaterial(itemId);
```

好处：UI 里没有一行游戏规则，改界面不会碰坏玩法；反过来改玩法不会弄乱界面。

5.4 输入系统：`InputSystem.ts`

键	作用
W A S D	移动
Shift	冲刺
空格	跳
E / 回车	交互（对话、砍树、挖矿、钓鱼、潜水、建造）
Tab / I	背包
J	日志
K	存档
F1	帮助
Esc	关面板 / 释放鼠标
Delete	重置存档（需二次确认）
鼠标左键	交互（没锁定鼠标时）或锁定鼠标（用来转视角）

两个讲究：

- **指针锁定（Pointer Lock）** —— 点一下画面，鼠标指针消失，鼠标移动变成"转视角"，这是所有第三人称游戏的做法；
- **一次性动作消费** —— `consume('interact')` 取到就删掉，保证"按一次 E 只做一件事"，不会因为按住而连砍十下。

5.5 交互判定：`InteractionSystem.ts`

决定"你按 E 会碰到什么"：

- **距离**：3.4 米内（`INTERACTION.range`）；
- **朝向**：你面朝的方向和目标方向的点积 > 0.2（避免背后的人也被触发）；
- 找到最近的合法目标 → 头顶弹出提示 → 按 E 触发对应行为。

六、玩法系统清单（`src/systems/`，20 个文件）

模块	职责	关键数字
`Game`	总导演、主循环、存档调度	2,400+ 行
`Player`	移动、跳跃、游泳、涉水、碰撞	走速 6.4，冲刺 ×1.75
`CameraController`	环绕相机、阻尼、避障	距离 7.2，FOV 62
`InputSystem`	键盘鼠标 → 离散动作	见上表

模块	职责	关键数字
`InteractionSystem`	找目标、算距离朝向	3.4 米
`CollisionSystem`	空间哈希 + 圆形碰撞体	格宽 8
`StaminaSystem`	体力、恢复、上限计算	上限 60，空腹减半
`SatietySystem`	饱食度四档：饱满/有点饿/很饿/虚弱	0.18/秒，见下表
`InventorySystem`	背包、单类上限	每类 20
`StorageSystem`	小屋的箱子	24 格 × 20
`CraftingSystem`	合成配方与材料检查	—
`BuildingSystem`	建造流程	—
`DecorationSystem`	小屋周边装饰	6 种
`InteriorSystem`	屋内家具与扩建	10 件
`MarketSystem`	买卖、工具等级与磨损、修理	工具 3 级
`TrustSystem`	好感度等级	0~4 级
`FishingSystem`	抛竿、咬钩、拉线判定	5 种鱼
`QuestSystem`	7 个任务、章节解锁	7 个
`TimeSystem`	时段、天气、每日轮换	一天 10 分钟
`SaveSystem`	3 个手动槽 + 自动存档、脏数据清洗	10 秒自动存

饱食度的四档（这是最像"养成"的一块）

饱食度	状态	体力恢复	移动速度	冲刺	干活
≥ 60	精神饱满	×1.25	×1.00	可以	可以
30~60	有点饿	×1.00	×0.92	可以	可以
10~30	很饿	×0.50	×0.85	不行	可以
0~10	虚弱	×0.35	×0.70	不行	**不行**

饿到"虚弱"时，砍树、挖矿、钓鱼会被拒绝，但**还能走路、还能摘浆果、还能吃东西** —— 不会卡死，只是很狼狈。

七、声音：现场合成的音乐

`src/audio/AudioSystem.ts`（640 行）**没有一个音频文件**，全部用 Web Audio 的振荡器和噪声实时合成：

- **3 种音乐情绪**：白天 / 夜晚 / 室内。跟着时段和"你在不在屋里"自动切换；
- **14 种音效**：脚步、砍击、挖掘、抛竿、咬钩、收获、金币、吃、放置、建造、错误、阶段变化……；

- **一个调度器**：每 100 毫秒提前排好接下来的音符（`schedule()` → `playStep()`），用 Web Audio 的精确时钟播放，不会因为主循环掉帧而走音；
- 音阶用大调 `[0,4,7]` 和小调 `[0,3,7]`，夜晚用小调，听着就"静"。

好处：一个音乐文件几百 KB，合成是 0 字节；而且想改节奏随时改数字。

八、存档：存在你自己的浏览器里

`src/systems/SaveSystem.ts`：

- 底层是 `localStorage`（浏览器自带的"本机小仓库"），键名 `lowpoly-island.save.v1`；
- **3 个手动槽 + 1 个自动存档**，每 10 秒自动存一次；
- `sanitizeSave()` 会把读出来的数据**逐字段校验**：不是数字的丢弃、超范围的不给、未知字段忽略 —— 防止手改存档或版本升级后把游戏弄崩；
- 存的是**状态**而不是"世界快照"：钱多少、背包里有什么、树被砍了哪几棵（靠资源 ID 记录）、任务做到哪、跟每个人好感多少。

因为世界是用**固定种子**生成的，读取时重新生成一次完全一样的岛，再把"砍掉的树"按 ID 标回去 —— 所以存档文件很小（几 KB），却能完整还原现场。

九、为什么能通过网页链接直接玩（核心问题）

这一章回答你的疑问。

9.1 浏览器自带三大能力

能力	浏览器提供什么	游戏的用法
3D 渲染	WebGL（GPU 接口）	Three.js 画地形、水、人物、树
界面	HTML + CSS + DOM	所有面板、按钮、文字
声音	Web Audio API	合成音乐和音效
存储	localStorage	存档

游戏需要的东西，浏览器全都自带。所以你不需要安装 Unity、不需要游戏引擎、不需要显卡驱动、不需要打包成 exe。

9.2 "没有服务器"意味着什么

这个游戏的全部逻辑都在**你打开网页时下载的那一个 HTML 文件**里：

- 不需要注册、不需要登录；
- 不需要联网（打开后不发出任何网络请求，唯一的 `https://` 是 three.js 着色器里的一行参考文献注释）；
- 关掉页面再打开，存档还在（因为存在你本机浏览器里）；
- 你的钱、你的进度**只在你自己的电脑上**，别人看不到。

所以"发个链接就能玩"这件事，在技术上是必然的：**它本来就是一段在浏览器里运行的程序。**

9.3 构建产物：一个自包含的 HTML

开发时是 20 多个源文件，交付时打包成**一个文件**。用两招：

第一招 —— `vite-plugin-singlefile`（构建时）：把 JS、CSS 全部**内联**进 HTML，一个外链都不留。

```
// vite.config.ts
const single = mode === 'single';
plugins: single ? [viteSingleFile()] : [],
build: { assetsInlineLimit: single ? 100_000_000 : 4096,   // 内联限额放到 100 MB
         rollupOptions: single ? { output: { format: 'iife', inlineDynamicImports: true } } : {} }
```

`format: 'iife'` + `inlineDynamicImports` 的意思是：**整包代码作为一个普通脚本执行，不带任何 import**。这样用 `file://` 直接双击打开也不会被浏览器的模块跨域策略拦住。

第二招 —— `scripts/postbuild-single.mjs`（构建后）：

```
html = html.replace('<script type="module" crossorigin>', '<script type="module">');
html = html.replace(/<link rel="modulepreload"[^>]*>/g, '');
// 再把 share/ 里的《试玩说明.txt》一起拷进输出目录
```

顺带做两件小事：去掉对内联脚本无意义的 `crossorigin` 属性；把玩家说明放在游戏旁边。

最终产物 `dist-single/index.html`：**727 KB，0 个外链，0 个 `import()`，0 个 `fetch()`**。

9.4 想变成"一个链接"的三种办法

方式	怎么做	适合谁
本地双击	直接双击 `index.html`	自己玩
本机开服务	`npx serve .` 然后访问 `http://localhost:3000`	局域网内给别人玩
传到静态托管	把 `index.html` 传到任意静态托管（GitHub Pages / 对象存储 / 自己的 Nginx），得到一个 `https://...` 链接	分享给所有人

第三种就是"点个链接直接玩"：因为整个游戏只有一个文件、没有后端，静态托管是**零成本**的 —— 托管方只是把这个文件原样发出去。

9.5 那我们为什么发的是 ZIP

因为中文即时通讯软件（微信、QQ）**会拦截 .html 文件**，发出去会被判成"危险文件"或者根本不让下载；ZIP 不会。

所以流程是：构建单文件 → 打包成 ZIP（附上《试玩说明.txt》）→ 发给朋友 → 朋友解压双击 `index.html` 就能玩。

这也是为什么每次改完都重新出一个带时间戳的 ZIP：**版本清晰、可以随时发最新版**。

十、工程实践：怎么保证它不出错

做法	目的	落地情况
TypeScript 严格模式	少写低级错误	`strict: true` + `noUncheckedIndexedAccess` 等 12 条
编译前检查	改完立刻知道有没有写错	`npm run typecheck`（`tsc --noEmit`）
单元测试	数值逻辑不靠手测	**289 个测试，23 个文件**
逻辑与渲染解耦	纯逻辑能在 Node 里直接测	`terrain.ts` 故意不 import three
种子确定性	同一台岛永远一样	`WORLD.seed = 20260921`
配置集中	改数值不碰逻辑	全部集中在 `src/config.ts`
数据驱动	加内容不写代码	物品/衣服/鱼/任务/商店/对话全在 `data/`

测试覆盖的都是"容易悄悄坏掉"的地方，例如：

- 体力代价：徒手砍树 25 点、石斧一级 14 点、二级 11 点、三级 8 点（越好的工具越省力）；
- 饱食度四档的阈值和效果；
- 卖东西不能产生套利（木板价 ≤ 2 × 木材价）；
- 钓鱼的咬钩与中鱼判定；
- 存档的脏数据清洗；
- 箱子存取、格子上限、JSON 往返。

十一、数据驱动：加内容不用改代码

`src/data/` 里的 9 个文件（2,082 行）就是游戏的"内容表"：

文件	内容
`items.ts`	物品名字、图标、用途（哪些能吃、恢复多少）
`npcs.ts`	三个村民：外形配色、性格、身份、商店偏好
`dialogue.ts`	全部台词：打招呼、闲聊、反应、任务提示
`shop.ts`	商店货架、价格、买家偏好、工具等级、废品价
`fish.ts`	5 种鱼：栖息地、出现时段、价值
`recipes.ts`	合成配方与材料
`clothing.ts`	9 件衣服与加成数值
`decorations.ts`	6 种装饰与摆放位置
`interior.ts`	家具与小屋扩建

想加一件新衣服？在 `clothing.ts` 里加 6 行，想加一句台词？在 `dialogue.ts` 里加 1 行 —— **不用碰任何逻辑代码。**

十二、如果你想自己做一遍：最小可行路线

按这个顺序做，每一步都能单独玩起来：

第 1 步 —— 一个能转的地形 写一个 `heightAt(x, z)` 函数（哪怕就一个正弦波），采样成网格，加一个相机。→ 已经有"一个岛"。

第 2 步 —— 一个能走的方块人 一个 `THREE.Group` 里放一个方块，读 WASD 移动，Y 轴贴地高度。→ 已经有"一个角色"。

第 3 步 —— 一个能按 E 的东西 世界里有几个箱子，检测距离和朝向，提示"按 E"，按了给一个方块。→ 已经有"交互"。

第 4 步 —— 一个能点的界面 用 `<div>` 做一个背包面板，点按钮调用游戏函数。→ 已经有"GUI"。

第 5 步 —— 打包 `vite build` + 内联插件，得到一个可以双击的 HTML。→ 已经有"能发给别人玩的东西"。

之后再慢慢加：昼夜与天气、音效、任务、好感度、存档、碰撞、实例化优化……

这个项目的真实顺序也差不多，只是每一步都多做了几层。

十三、文件地图

```
lowpoly-island/
├─ index.html              页面骨架：canvas + 14 个面板 + HUD
├─ style.css               全部界面样式（1,000+ 行）
├─ package.json            依赖与脚本（dev / build / build:single / test）
├─ vite.config.ts          构建配置：普通站点 vs 单文件
├─ vitest.config.ts        测试配置
├─ tsconfig.json           TypeScript 严格配置
├─ scripts/
│  └─ postbuild-single.mjs 构建后处理：内联脚本修正 + 拷贝说明
├─ share/
│  └─ 试玩说明.txt          随游戏分发的玩家说明
├─ src/
│  ├─ main.ts              入口：36 行
│  ├─ config.ts            全部可调参数（世界/玩家/相机/交互/经济/存档）
│  ├─ data/                内容表（9 个文件）
│  ├─ world/               地形/水面/道具/资源/建筑（7 个文件）
│  ├─ entities/            人物模型与动画（4 个文件）
│  ├─ systems/             玩法系统（20 个文件）
│  ├─ ui/GameUI.ts         界面渲染（1 个文件，1,326 行）
│  ├─ audio/AudioSystem.ts 声音合成（1 个文件，640 行）
│  └─ utils/               噪声/随机/数学（3 个文件，188 行）
└─ tests/                  23 个测试文件，289 个测试
```

十四、名词小词典

词	意思
Three.js	浏览器里的 3D 引擎，负责把三角形画到屏幕上
WebGL	浏览器提供的 GPU 绘图接口，Three.js 底层调它
场景 Scene	装所有物体的"世界容器"
相机 Camera	决定你从哪看世界
网格 Mesh	一个能画出来的几何体（树、人、地形）
材质 Material	决定网格长什么样（颜色、反光、是否受光）
着色器 Shader	写在 GPU 上跑的小程序（天空渐变、水面波动都是它）
顶点色 Vertex Color	直接存在顶点里的颜色，用来代替贴图
实例化 InstancedMesh	同一个形状画很多遍，只花一次绘制开销
GLSL	写着色器用的语言
扁平着色 Flat Shading	每个三角形用同一个法线，做出块块分明的低多边形感
指针锁定 Pointer Lock	锁定鼠标、隐藏指针，用鼠标移动转视角
localStorage	浏览器自带的本机小仓库，存存档用
Vite	前端构建工具：开发时热更新，发布时打包压缩
TypeScript	带类型检查的 JavaScript，写错了编译就报错
Vitest	测试框架，跑那 289 个测试
确定性 Determinism	同一个种子永远生成同一个世界，存档才能记住"哪棵树被砍了"
fBm 分形噪声	把多种频率的噪声叠起来，得到自然的起伏

文档依据 `lowpoly-island` 项目的实际源码整理，统计数字与代码行数取自 2026-10-22 的仓库状态。